Programski prevodioci

13 silazno parsiranje

25-26

Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad

Parsiranje

- Parseru stiže niz tokena
- Izvodi se sintaksna struktura programa
- Transformiše se običan string u sintaksni model
- Cilj: dokazati da se ulazni program može izvesti na osnovu pravila gramatike jezika
- Na osnovu pravila izvođenja se gradi stablo parsiranja, često implicitno

Parsiranje od dna ka vrhu // Uzlazno parsiranje // bottom-up parsing

Pokušavamo da od ulaznog niza simbola redukcijom dobijemo početni pojam

Akcije: Desna strana pravila se zamenjuje levom
Stablo prasiranja: gradi se od listova prema vrhu

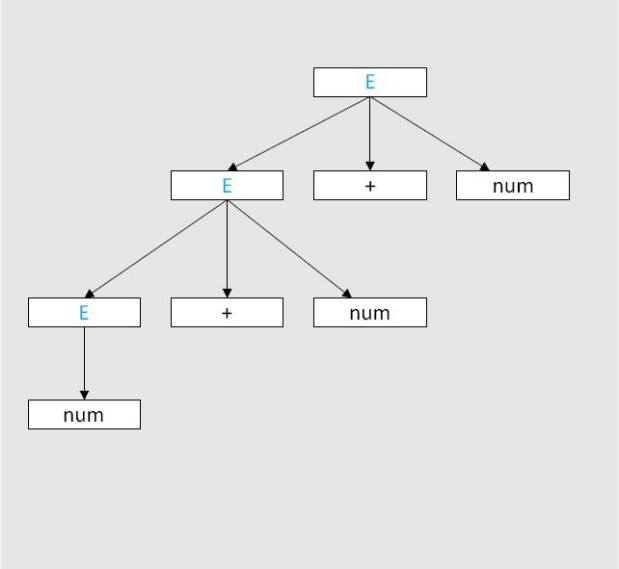
Parsiranje od vrha ka dnu // Silazno parsiranje // top-down parsing

Pokušavamo da od početnog pojma izvedemo ulazni niz simbola prateći pravila produkcije

Akcije: Leva strana pravila se zamenjuje desnom
Stablo prasiranja: gradi se od vrha prema listovima

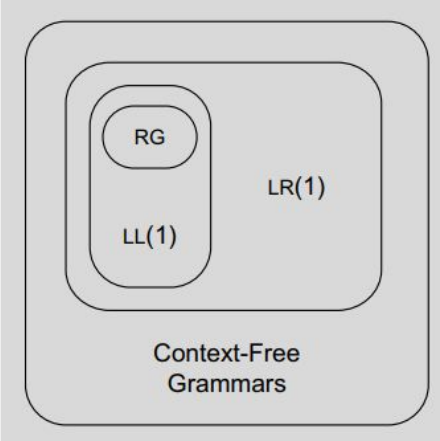
E → num
E → E + num

5 + 3 + 1
w = num + num + num



E
⇒ E + num
⇒ E + num + num
⇒ num + num + num

Različiti podskupovi CFG gramatika (jezika) - različite tehnike parsiranja



LR(1)

- bison
- bottom-up parsiranje
- LALR - nedvosmislene gramatike koje se mogu parsirati uzlazno sa jednim preduvidnim (LA) tokenom

Silazno parsiranje

- Kreće se od početnog neterminala, kreira se vrh stabla

- U svakom koraku se bira čvor u trenutno izgrađenom stablu koji predstavlja neki neterminal
- Za njega se bira pravilo i razvija podstablo koje odgovara tom pravilu

- Ponavlja se sve dok
    - se ne dođe do listova (terminali, tokeni) čiji niz odgovara ulaznom nizu → OK ILI
    - dok se ne otkrije problem:

        - parser nije dobro odlučio prilikom odabira pravila → vraćanje unazad (backtrack)
        - sve opcije su isprobane - nije validan ulaz → greška

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * num$
4. $T \rightarrow num$

num + num * num

```
     E
(2)  T
(4)  num
error / backtrack
```

```
     E
(2)  T
(3)  T * num
(4)  num * num
error / backtrack
```

```
     E
(1)  E + T
(2)  T + T
(4)  num + T
(3)  num + T * num
(4)  num + num * num
ok
```

Backtracking:

- fokus (obradu) vratiti na roditeljski čvor
- izbaciti generisano podstablo
- ako je ostalo neisprobanih pravila za neterminal koji se obrađuje - izabrati novo
- ako nije ostalo neisprobanih pravila – povratak na nivo iznad
- ako su iscrpljene sve mogućnosti (vrh koji odgovara početnom pojmu i sva njegova pravila) → ulazni string nije validan

Obratiti pažnju

- mora ce rekonstruisati i obrađeni deo ulaznog stringa
- potrebno je, prilikom uklanjanja podstabla, sve terminale “vraćati” kao deo ulaznog stringa

```
root ← node for the start symbol, S;
focus ← root;
push(null);
word ← NextWord();

while (true) do;
   if (focus is a nonterminal) then begin;
      pick next rule to expand focus (A → β1,β2,...,βn);
      build nodes for β1,β2...βn as children of focus;
      push(βn, βn−1, ..., β2);
      focus ← β1;
   end;
   else if (word matches focus) then begin;
      word ← NextWord();
      focus ← pop()
   end;
   else if (word = eof and focus = null)
      then accept the input and return root;
      else backtrack;
end;
```

- *pick next rule to expand focus (A → β1 ,β2 ,...,βn);*

Kad u jednom momentu posmatramo jedan neterminal (*A*) za koji postoji više pravila (*β1 , β2 ,...,βn)* mora postojati heuristika za izbor pravila.

Koja je heuristika?

- prvo pravilo koje se pojavljuje u gramatici
- “najkraće/najduže” pravilo (šta je najkraće?)
- slučajni izbor pravila?
- nešto drugo

Šta je dobra heuristika?

- Ona sa kojom bismo postigli što manje vraćanja unazad.
- Ako bismo svaki put izabrali tačno ono pravilo koje treba → idealno, direktno parsiranje, bez vraćanja

```
1.   E → E + T
2.   E → T
3.   T → T * num
4.   T → num
```

```
num + num * num
```

Pretpostavimo da je algoritam: probaj pravila redom kako se pojavljuju u gramatici.

```
     E
(1)  E + T
(1)  E + T
(1)  E + T
...
```

Nikad se ne generiše terminal i da, konačno, možemo da uporedimo prvi token (num)

→ leva rekurzija je problem

Eliminacija leve rekurzije

$$A \rightarrow A\alpha \mid \beta$$

$$\beta\alpha\alpha\alpha\alpha$$

levo rekurzivno

$A \rightarrow A\alpha \mid \beta$ $\longrightarrow$

$A \rightarrow \beta R$

$R \rightarrow \alpha R \mid \varepsilon$

levo rekurzivno

uveden novi pojam
desno rekurzivno

$\beta\alpha\alpha\alpha\alpha$

$$A \rightarrow A\alpha \mid \beta \longrightarrow \begin{array}{l} A \rightarrow \beta R \\ R \rightarrow \alpha R \mid \varepsilon \end{array}$$

$$\begin{array}{ll} E & \rightarrow E + T \\ & \mid T \end{array} \longrightarrow \begin{array}{ll} E & \rightarrow T\ E' \\ E' & \rightarrow +\ T\ E' \\ & \mid e \end{array}$$

$A \rightarrow A\alpha \mid \beta$ $\longrightarrow$

$A \rightarrow \beta R$

$R \rightarrow \alpha R \mid \varepsilon$

$E \rightarrow E + T \mid T$ $\longrightarrow$

$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E' \mid e$

```
var_list → variable
         | var_list variable
```

```
var_list  → variable var_list1
var_list1 → variable var_list1
          | ε
```

α == β

$A \rightarrow A\alpha \mid \beta$ $\longrightarrow$ $A \rightarrow \beta R$

$R \rightarrow \alpha R \mid \varepsilon$

```
1.   E    → E + T
2.        | T
3.   T    → T * num
4.        | num
```

```
1.   E    → T E'
2.   E'   → + T E'
3.        | ε
4.   T    → num T'
5.   T'   → * num T'
          | ε
```

U prethodnim primerima postoji direktna leva rekurzija.
Lako se uočava, ručno se može ukloniti.

Može postojati indirektna leva rekurzija.
Kroz više pravila se pojavljuje lanac.

$$\alpha \rightarrow \beta$$
$$\beta \rightarrow \gamma$$
$$\gamma \rightarrow \alpha\boldsymbol{\delta}$$

$$\alpha \rightarrow^* \alpha\boldsymbol{\delta}$$

Zadatak

- pronaći sve indirektne leve rekurzije
- zameniti ih direktnim levim rekurzijama
- zameniti direktne leve rekurzije desnim

```
impose an order on the nonterminals, A1, A2, ..., An

for i ← 1 to n do;
    for j ← 1 to i - 1 do;
        if ∃ a production Ai→Ajγ
            then replace Ai→Ajγ with one or more
                 productions that expand Aj
    end;
    rewrite the productions to eliminate
         any direct left recursion on Ai
end;
```

∀ k < i, no production expanding Ak has Al in its rhs, for l < k.

Prediktivna gramatika / Predictive grammar

CFG gramatika gde silazno parsiranje sa najlevljim izvođenjem uvek bira ispravno pravilo koristeći k preduvidnih (LA) terminala

Na osnovu narednih, ulaznih, tokena se određuje koje pravilo će se birati za nastavak.

→ backtrack-free parsiranje

LL(k)
LL(1)

```
1.   E    → T E'
2.   E'   → + T E'
3.        | ε
4.   T    → num T'
5.   T'   → * num T'
          | ε
```

Ako vidim da dolazi + znam koje pravilo da biram za E'

Za svaki neterminal α se određuje skup FIRST(α) koji sadrži sve terminale koji se mogu pojaviti na početku prilikom izvođenja α.

Elementi FIRST skupa za tokene, praznu reč i eof su oni sami.

- Posmatraju se desne strane pravila
- Ako desne strane počinju terminalima – oni ulaze u skup početnih
- Ako desne strane počinju neterminalima – dodati i njihove početne simbole u skup

```
type → "int" | "unsigned"
FIRST(type) = {"int" | "unsigned“ }
```

```
variable → type identifier
FIRST(variable) → {"int" | "unsigned“}
```

```
statement          → if_statement
                       | return_statement
if_statement       → “if” “(” rel_exp “)” statement
return_statement   → “return” num_exp “;”


FIRST(statement) = { if, return }
FIRST(if_statement) = { if }
FIRST(return_statement) = { return }
```

```
E    →   T E'
E'   →   + T E'
     |   - T E'
     |   ε

T    →   num T'
T'   →   * num T'
     |   / num T'
     |   ε
```

```
FIRST(ε) = { ε }
FIRST(num) = { num }

FIRST(/ num T') = {/}
FIRST(* num T') = {*}
FIRST(T') = {/, *, ε}
FIRST(T) = {num}

FIRST(- T E') = {-}
FIRST(+ T E') = {+}
FIRST(E') = {-, +, ε}

FIRST(E) = {num}
```

Za niz simbola s, FIRST(s) je unija skupova svih FIRST skupova simbola sa desne strane, od prvog simbola do simbola čiji FIRST skup ne sadrži praznu reč.

Prazna reč pripada skupu FIRST(s) ako ona pripada svakom simbolu sa desne strane.

Algoritam

Za svaki neterminal X:

- ako postoji izvođenje $X \rightarrow^* \varepsilon$, dodati ε u FIRST(X)

- za pravilo $X \rightarrow Y_1\ Y_2 \ldots Y_k$ sve simbole iz skupa FIRST(Y_1) dodati u FIRST(X)
  - ako postoji izvođenje $Y_1 \rightarrow^* \varepsilon$ onda sve simbole iz skupa FIRST(Y_2) dodati u FIRST(X)
  - ukoliko je potrebno, isti postupak primenjivati na $Y_2, \ldots Y_k$

```
for each α ∈ (T ∪ eof ∪ ϵ) do;
    FIRST(α) ← α;
end;
for each A ∈ NT do;
    FIRST(A) ← ∅ ;
end;
while (FIRST sets are still changing) do;
   for each p ∈ P, where p has the form A→β do;
       if β is β1β2...βk, where βi ∈ T ∪ NT, then begin;
         rhs ← FIRST(β1) − {ϵ};
         i ← 1;
         while (ϵ ∈ FIRST(βi) and i ≤ k-1) do;
             rhs ← rhs ∪ (FIRST(βi+1) − {ϵ}) ;
             i ← i + 1;
         end;
       end;
      if i = k and ϵ ∈ FIRST(βk)
           then rhs ← rhs ∪ {ϵ};
      FIRST(A) ← FIRST(A) ∪ rhs;
   end;
end;
```

```
E    →    T E'
E'   →    + T E'
     |    - T E'
     |    ε

T    →    num T'
T'   →    * num T'
     |    / num T'
     |    ε
```

```
FIRST(ε) = { ε }
FIRST(num) = { num }

FIRST(/ num T') = {/}
FIRST(* num T') = {*}
FIRST(T') = {/, *, ε}
FIRST(T) = {num}

FIRST(- T E') = {-}
FIRST(+ T E') = {+}
FIRST(E') = {-, +, ε}

FIRST(E) = {num}
```

3 + 5

- Kreće se od pravila 1 (E). Nema odlučivanja, samo je jedno pravilo.
- pravilo 5 (T). Nema odlučivanja. 3 → num.
- T' odluka: 6., 7. ili 8.
  - LA je +
  - + ne pripada FIRST(T')
  - šta dalje? Možda je greška, a možda se treba vratiti i nastaviti gde smo stali kod E.

Problem je ε čiji skup ne sadrži nijedan terminal.
Da bismo mogli da razlikujemo sintaksne greške od “nastavka” uvodi se skup FOLLOW.

Za svaki simbol α se određuje skup FOLLOW(α) koji sadrži sve terminale koji mogu da se pojave na početku nakon izvođenja α.

```
E    →   T E’
E’   →   + T E’
     |   - T E’
     |   ε

T    →   num T’
T’   →   * num T’
     |   / num T’
     |   ε
```

```
FOLLOW(T’) = {+, -, eof}
FOLLOW(T) = {+, -, eof}
FOLLOW(E’) = {eof}
FOLLOW(E) = {eof}
```

```
FOLLOW(num) = {+, -, *, / }
```

Algoritam

Za svaki neterminal X

- Ako je X početni pojam gramatike, dodati eof u FOLLOW(X)

- Za svako pravilo u kome se X pojavljuje i sa desne strane:
  - U pravilima u kojima se iza X nalaze drugi pojmovi i/ili simboli, na primer $Y \rightarrow \alpha X \beta$, dodati FIRST(β)\{ε} u FOLLOW(X). Ako ε postoji u FIRST(β), tada dodati FOLLOW(Y) u FOLLOW(X).

  - U pravilima u kojima se X nalazi na poslednjem mestu, na primer $Y \rightarrow \alpha X$, dodati FOLLOW(Y) u FOLLOW(X).

```
for each A ∈ NT do;
    FOLLOW(A) ← ∅ ;
end;

FOLLOW(S) ← {eof} ;

while (FOLLOW sets are still changing) do;
   for each p ∈ P of the form A → β1β2···βk do;
       TRAILER ← FOLLOW(A) ;

       for i ← k down to 1 do;
           if βi ∈ NT then begin;

               FOLLOW(βi) ← FOLLOW(βi) ∪ TRAILER ;

               if ϵ ∈ FIRST(βi)
                  then TRAILER ← TRAILER ∪ (FIRST(βi) − ϵ) ;
                  else TRAILER ← FIRST(βi) ;
           end;
           else TRAILER ← FIRST(βi) ;     // is {βi}
       end;
   end;
end;
```

```
E     →    T E'
E'    →    + T E'
      |    - T E'
      |    ε

T     →    num T'
T'    →    * num T'
      |    / num T'
      |    ε
```

```
FIRST(/ num T') = {/}
FIRST(* num T') = {*}
FIRST(T') = {/, *, ε}
FIRST(T) = {num}

FIRST(- T E') = {-}
FIRST(+ T E') = {+}
FIRST(E') = {-, +, ε}

FIRST(E) = {num}
```

```
FOLLOW(T') = {+, - }
FOLLOW(T) = {+, - }
FOLLOW(E') = {}
FOLLOW(E) = {}
```

3 + 5

- Kreće se od pravila 1 (E). Nema odlučivanja, samo je jedno pravilo.
- pravilo 5 (T). Nema odlučivanja. 3 → num.
- T' odluka: 6., 7. ili 8.
  - LA je +
  - + ne pripada FIRST(T'); + pripada FOLLOW(T') i FIRST(+TE') → pravilo 2.

```
E     →    T E'
E'    →    + T E'
      |    - T E'
      |    ε

T     →    num T'
T'    →    * num T'
      |    / num T'
      |    ε
```

```
FIRST(/ num T') = {/}
FIRST(* num T') = {*}
FIRST(T') = {/, *, ε}
FIRST(T) = {num}

FIRST(- T E') = {-}
FIRST(+ T E') = {+}
FIRST(E') = {-, +, ε}

FIRST(E) = {num}
```

```
FOLLOW(T') = {+, - }
FOLLOW(T) = {+, - }
FOLLOW(E') = {}
FOLLOW(E) = {}
```

- Da je bilo 3 * 5 → LA je * → pravilo 6
- Da je bilo 3 5 → num ne pripada FOLLOW(T') → sintaksna greška
- Da je bilo 3 → LA je eof → kraj, ok

- Sledbenički skup olakšava i oporavak od grešaka u toku kompajliranja (panic mode error-recovery)

- omogućava da se nakon otkrivanja pogrešnog simbola ignorišu simboli koji se u ulaznom nizu nalaze između pogrešnog simbola i simbola koji pripada sledbeničkom skupu

- sinhronizacioni skup

Generalizacija

$\texttt{FIRST}^+(\alpha \rightarrow \boldsymbol{\beta}) = \texttt{FIRST}(\boldsymbol{\beta})$, ako $\texttt{FIRST}(\boldsymbol{\beta})$ ne sadrži ε
$\texttt{FIRST}^+(\alpha \rightarrow \boldsymbol{\beta}) = \texttt{FIRST}(\boldsymbol{\beta}) \cup \texttt{FOLLOW}(\alpha)$, u suprotnom.

backtrack free, predictive - FIRST^+ skupovi su disjunktni

$$\begin{array}{lll} \alpha & \rightarrow & \boldsymbol{\beta}_1 \\ & \mid & \boldsymbol{\beta}_2 \\ & \mid & \boldsymbol{\beta}_3 \\ & \vdots & \\ & \mid & \boldsymbol{\beta}_n \end{array}$$

$\texttt{FIRST}^+(\alpha \rightarrow \boldsymbol{\beta}_i) \cap \texttt{FIRST}^+(\alpha \rightarrow \boldsymbol{\beta}j) = \varnothing$, za svako i, j

$$
\begin{array}{lll}
A & \rightarrow & x\boldsymbol{\beta}_1 \\
 & | & x\boldsymbol{\beta}_2 \\
 & \dots & \\
 & | & x\boldsymbol{\beta}_n \\
 & | & \boldsymbol{\gamma}_1 \\
 & | & \boldsymbol{\gamma}_2 \\
 & \dots & \\
 & | & \boldsymbol{\gamma}_m
\end{array}
$$

FIRST$^+$ nisu disjunktni

Leva faktorizacija

$$
\begin{array}{lcl}
A & \rightarrow & x\beta_1 \\
 & | & x\beta_2 \\
 & \dots & \\
 & | & x\beta_n \\
 & | & \gamma_1 \\
 & | & \gamma_2 \\
 & \dots & \\
 & | & \gamma_m
\end{array}
\quad \longrightarrow \quad
\begin{array}{lcl}
A & \rightarrow & xB \\
 & | & \gamma_1 \\
 & | & \gamma_2 \\
 & \dots & \\
 & | & \gamma_m \\
 & & \\
B & \rightarrow & \beta_1 \\
 & | & \beta_2 \\
 & \dots & \\
 & | & \beta_n
\end{array}
$$

Izdvajanje zajedničkog dela i uvođenje novog terminala
Dobijaju se disjunktni FIRST$^+$

Uklanjanje leve rekurzije + leva faktorizacija

→ Alati za eventualno postizanje dobijanja gramatike gde nema potrebe za vraćanjem unazad

Nemamo algoritam za utvrđivanje da li postoji backtrack-free gramatika za proizvoljan CFG jezik.

Recursive descent parser (rekurzivni spust)

- skup rekurzivnih funkcija
- sa svaki neterminal postoji odgovarajuća funkcija
- svaka od ovih funkcija prepoznaje odgovarajući neterminal u ulaznom tekstu

Algoritam:

- Izvršavanje počinje od prvog, početnog, neterminala
- Svaka funkcija, na osnovu FIRST+ skupa odlučuje o pravilu sa desne strane.
- Za izabrano pravilo:
  - ako je terminal – upoređivanje sa LA i pomeranje
  - ako je neterminal - poziv odgovarajuće funkcije

```
E    →   T E'
E'   →   + T E'
     |   - T E'
     |   ε

T    →   num T'
T'   →   * num T'
     |   / num T'
     |   ε
```

```
EPrim()
     if(LA = + OR -)
          LA = nextToken
          if (T())
               return Eprim()
          else
               return false
     else
          if (LA = eof)
               return true
          else
               report Syntax error
               return false
```

$$
\begin{array}{lcl}
E & \rightarrow & T\ E' \\
E' & \rightarrow & +\ T\ E' \\
 & | & -\ T\ E' \\
 & | & \varepsilon \\
 & & \\
T & \rightarrow & \text{num}\ T' \\
T' & \rightarrow & *\ \text{num}\ T' \\
 & | & /\ \text{num}\ T' \\
 & | & \varepsilon
\end{array}
$$

```
Main()
      LA = nextToken()
      if (E())
            if LA == eof
                  success
            else
                  try error recovery
E()
      if (T())
            return Eprim()
      else
            return false
EPrim()
      if(LA = + OR -)
            LA = nextToken
            if (T())
                  return Eprim()
            else
                  return false
      else
            if (LA = eof)
                  return true
            else
                  report Syntax error
                  return false
...
```

- Prikazan je okvir za ručno generisanje RD parsera
- programer piše funkcije za svaki pojam u gramatici
- jednostavno i pogodno za manje složene gramatike

Moguće je kreirati i parser generator.

Na osnovu gramatike se prvo kreira tabela
- redovi su neterminali
- kolone su terminali
- za svaki element tabele se upiše pravilo koje odgovara situaciji kad je odgovarajući terminal LA

Postoji funkcija koja na osnovu bilo koje tabele vrši parsiranje.

```
build FIRST, FOLLOW, and FIRST+ sets;

for each nonterminal A do;
    for each terminal w do;
        Table[A ,w] ← error;
    end;
    for each production p of the form A → β do;
       for each terminal w ∈ FIRST+(A → β) do;
           Table[A ,w] ← p;
           end;
       if eof ∈ FIRST+(A → β)
          then Table[A ,eof] ← p;
   end;
end;
```

```
word ← NextWord( );
push eof onto Stack;
push the start symbol, S, onto Stack;
focus ← top of Stack;
loop forever;
   if (focus = eof and word = eof)
       then report success and exit the loop;
   else if (focus ∈ T or focus = eof) then begin;
      if focus matches word then begin;
         pop Stack;
         word ← NextWord( );
      end;
      else report an error looking for symbol at top of stack;
   end;
   else begin; /* focus is a nonterminal */
      if Table[focus,word] is A → B1B2···Bk then begin;
         pop Stack;
         for i ← k to 1 by -1 do;
           if (Bi ≠ ε)
              then push Bi onto Stack;
         end;
      end;
      else report an error expanding focus;
    end;
    focus ← top of Stack;
end;
```

Rezime

Cilj:

- nema dvosmislenost
- nema leve rekurzije
- ima jednoznačne početne skupove

Većina konstrukta u programskim jezicima se može izraziti prediktivnim gramatikama.
Često se koristi za ručno pisane parsere sa eventualno dodatnim modifikacijama za specijalne slučajeve.

Njaveća mana je, zapravo, to što ne sme da ima levu rekurziju, a ona je za mnoge stvari prirodnija.

Algoritmi su preuzeti iz knjige:
Cooper, K. and Torczon, L., 2011. Engineering a compiler. Elsevier.

Pogledati sada:

PEP 617 – New PEG parser for CPython

LL(1) → PEG